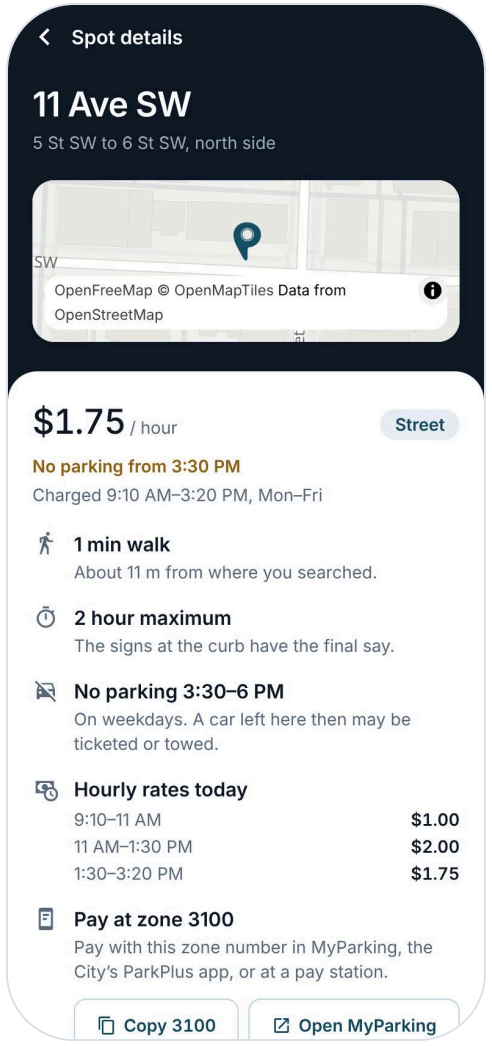
Case study

ParkAid

A mobile app for finding street parking in downtown Calgary. It uses the City of Calgary's open data for parking zones, rates and restrictions.

Role	Team member in a five-person SAIT capstone. After the capstone, I continued the project on my own, using AI coding tools.
Timeline	Capstone in Year 2, Semester 2. Follow-up work from September 2025 to November 2026.
Platforms	iOS, Android and web, from one React Native (Expo) codebase.
Tools	React Native, Expo, React Navigation, react-native-maps, MapLibre GL JS, Node.js, Express, PostgreSQL with PostGIS, Supabase, Render, Jest.
Links	Source code · Demo recordings · Figma file



Spot details for one block on 11 Ave SW.

Project summary

The problem	The capstone version showed prices that did not match the City's rates. It used raw City addresses, listed the same block several times, and had no working map on the web.
My role	I was one of five team members in the capstone. Afterward, I continued the project on my own. Claude Code, an AI coding tool, wrote most of the code from March 2026, working from my instructions.
What I changed	The app now reads the City's rates by time of day and its rush-hour restrictions, and shows each block once. Each spot shows its block, side of the street, zone, current rate and any restriction.
Results	Rates and restrictions match the City's data for the tested blocks. The opening map view has 15 pins instead of 45 overlapping ones. 170 automated tests pass. No usability testing with drivers has been done yet.

Contents

The problem	What the capstone version got wrong	3
My role and process	Who did what, and how each review round worked	4
Design changes	The Home screen, before and after	5
Map and filtering	How pins, sorting and filters work	6
Technical work	System overview, my front-end work, challenges	7
Testing	What was tested, results, and what was not tested	8
Outcome and links	What changed, next steps, source code, demo, Figma	9

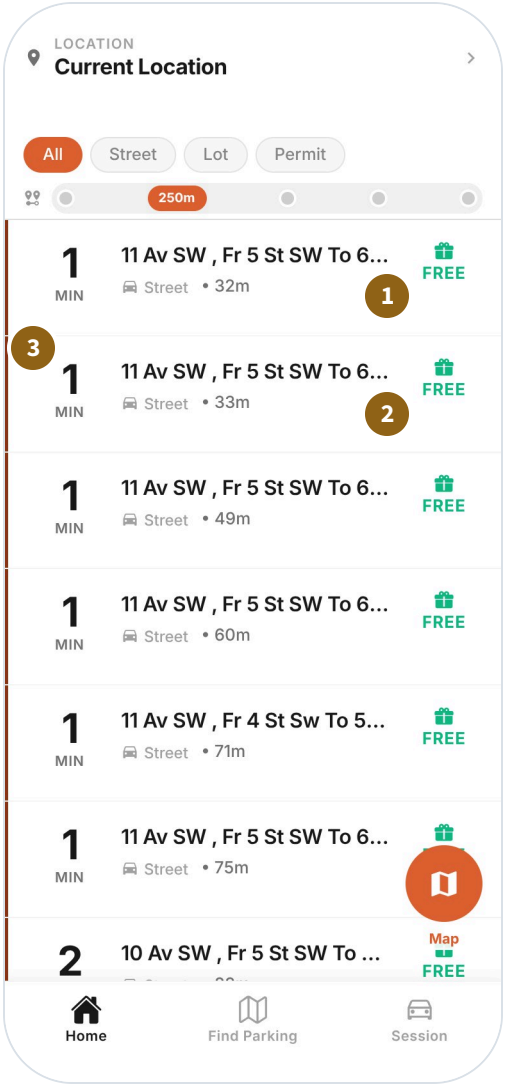
What the capstone version got wrong

Downtown parking rules change by block, side of the street and time of day. The City of Calgary publishes these rules as open data, but the capstone code from October 2025 did not use them correctly.

1. **Wrong prices.** The price came from a lookup on the zone code, not from the City's rates. Blocks on 11 Ave SW, which the City charges for on weekdays, showed FREE. Zone 1684 showed $6.00 an hour; the City charges $2.00 to $2.25.
2. **Raw addresses.** Rows used the City's address format, for example "11 Av SW , Fr 5 St SW To 6 St SW".
3. **Repeated rows.** The City splits one stretch of a block into several segments, and each segment appeared as its own row.

Not shown here: the web version displayed a static map image, and the parking session screen was a demo where the user chose the rate.

The numbers match the markers on the screen →



My role and how the work was done

During the capstone

A SAIT project with a team of five. Together we built the base app: a list of nearby spots, a map, a parking session screen, an Express API and an import of the City's parking data. This repository starts from that finished code (first commit September 29, 2025), so the team's own commits are not in it.

After the capstone

- Deployed the backend on Render with a Supabase database.
- Replaced the zone-code prices with the City's rates by time of day.
- Added rush-hour restrictions to the list, the spot details and the session.
- Merged repeated block segments in the API.
- Redesigned the screens and added a working map to the web version.
- Ran three rounds of design review and fixed the issues found.

Steps in each review round

1. **Capture.** Run the web build with City data for a fixed day and time, and take a screenshot of every screen.
2. **Review.** Check usability, wording, color contrast, keyboard use and screen sizes.
3. **Prioritize.** I chose the order of the fixes. In the last round: rates and restrictions first.
4. **Fix.** One change per commit.
5. **Verify.** Run the tests, check the web build at three screen widths, and build the Android bundle.

Use of AI tools

Up to October 28, 2026, Claude Code wrote 101 of the repository's 128 commits, working from my instructions. The other 27 are under my own accounts; a few of those came from an OpenAI Codex branch. I decided what to build and fix, and in what order.

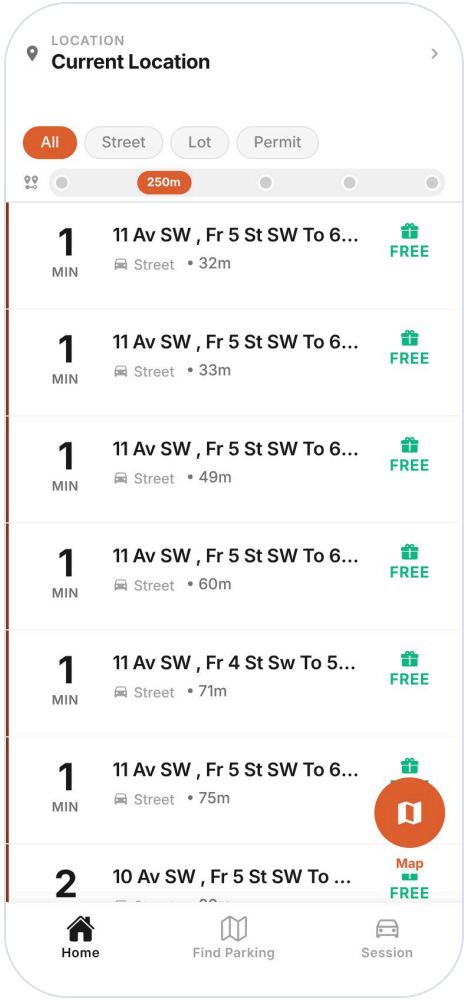
Design changes on the Home screen

Same blocks, same day and time (a Tuesday, 1:40 PM). The capstone version is on the left, the current version on the right.

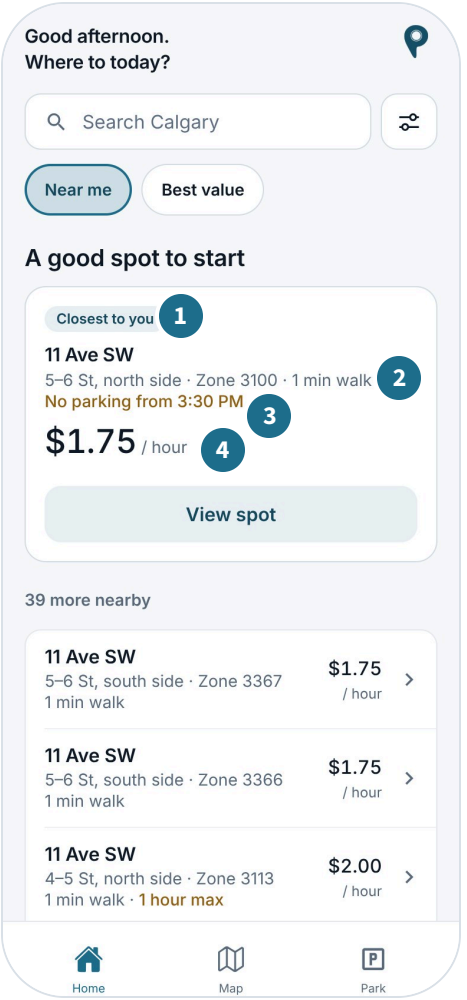
1. **Suggested spot.** The app picks one spot to start with and says why, for example “Closest to you”.
2. **Block, side and zone.** Written the way the street signs show them: “5–6 St, north side · Zone 3100”.
3. **Restrictions.** A rush-hour rule is shown in amber: “No parking from 3:30 PM”.
4. **Current rate.** The City’s rate for the current hour, per hour.

Other screens: Spot details lists the day’s rates and the zone to pay with. The Park screen shows the end time and the cost before a session starts, and stops the length before a rush-hour restriction.

Before



After



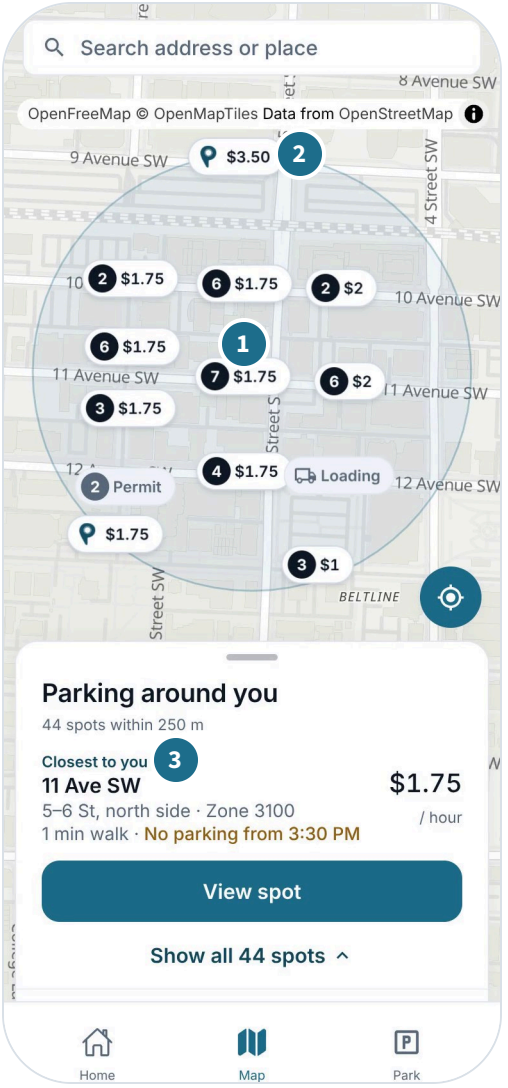
Map and filtering

Downtown, up to 45 parking blocks can be within 250 m of a driver. The map groups pins that would overlap, so each pin can be read.

1. **Group pin.** Shows how many spots it contains and the lowest rate. Tapping it zooms in until the spots separate.
2. **Price pin.** One spot with its current rate. A selected pin turns dark.
3. **Bottom sheet.** Shows the suggested spot. “Show all 44 spots” opens the full list.

Sorting and filters

- **Near me** sorts by walking time, then price. **Best value** sorts by the current rate, then distance.
- Filters: parking type (street, lot, parkade, permit) and distance (150 m to 1 km). The list and the map use the same filters.
- In both sort orders, short stays, blocks with no parking at the moment, and permit-only blocks are listed lower.



Technical work

City of Calgary open data
Parking zones, rates, restrictions

→

Import script
Loads four datasets into one table

→

PostgreSQL + PostGIS
Supabase; finds nearby spots

→

Express API
Render; merges block segments

→

React Native app
iOS and Android: Google Maps. Web: MapLibre with OpenStreetMap.

My front-end responsibilities

- **Screens and navigation:** Welcome, Home, Map, Spot details, Park and Session, with React Navigation.
- **Shared components:** spot rows, map pins and group pins, payment actions.
- **App logic in tested functions:** rates, rush-hour checks, ranking and pin grouping.
- **Platform code:** a separate web map file (MapLibre) and a smaller pin for Android.
- **State:** a session context for the timer, hooks for location and nearby spots, cached responses.
- **Accessibility:** screen reader labels, Reduce Motion support, keyboard order on the web.

Technical challenges

Challenge	Solution
Rates change during the day	Turned the City's rate bands into time ranges. Costs count only the hours the City charges.
Rush-hour restrictions	Checked each restriction against the current time and the end of a planned stay.
Repeated segments	The API merges rows for the same stretch of a block: 277 of 1,178 rows were repeats.
Crowded map	Grouped pins by their distance on screen at the current zoom, on both map libraries.
Android map markers	Android draws custom markers into a 100 × 100 px image, which cut off price labels, so Android uses a smaller badge.

Testing and results

Area	How it was checked	Result
Unit tests	Jest tests for rates, restrictions, ranking, grouping and wording (app); row merging, geometry and zone rates (backend)	152 app tests and 18 backend tests pass (November 2, 2026)
Real data	1,672 City records within 1.5 km of 11 Ave SW and 5 St SW, served through the backend's row code	Used for all screenshots and recordings
Map density	Counted pins and overlaps on the opening web map view	45 pins (44 overlapping) reduced to 15 pins (4 touching)
Color contrast	WCAG contrast ratio of text on selected rows	Gray text from 4.23:1 to 4.74:1 (AA minimum is 4.5:1); amber text 4.63:1
Keyboard (web)	Tabbed through the map screen	Map pins are skipped, since the list has the same spots; search is the first stop
Small screens	Web build at 360 × 640	"Add time" and "End session" moved from below the screen (676 and 771 px) to 263 and 358 px
Builds	Web build checked at 360, 390 and 1280 px wide; Android bundle built	Both build without errors

Not tested yet

- Usability testing with drivers
- Testing on a physical phone
- Database-backed test suites, which need a database connection
- Query speed

Outcome, next steps and links

What changed

- Prices come from the City’s rates for the current hour.
- Rush-hour restrictions are shown, and session lengths stop before them.
- Each block appears once, with its side of the street and zone.
- The web version has a working map with grouped pins.
- The backend is deployed and needs one setting to run.

Next steps

- Run usability tests with five drivers downtown.
- Test on a physical iPhone and Android phone.
- Load the new data import into the live database.
- Finish the Figma file: session screens and prototype links.

Links

Source code	github.com/ginesbal/parkaid
Demo	Seven screen recordings of the real build: 1 Finding parking, 2 The map, 3 Spot details, 4 Parking, 5 Rush hour, 6 Details while parked, 7 On a laptop.
Figma file	Variables, icons, 11 components and 5 screens, rebuilt from the code. Session screens and links are not finished.
LinkedIn	linkedin.com/in/ehrlbalquin

This case study was drafted with Claude Code. Its numbers come from the repository and from test runs.